Practical 1

Part A:

Aim:

1. Vending machine (VendingMachine.java, main in this file)

(a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.

(b) In main, set a snack price of 15 and a running total of 0; create a Scanner.

(c) Loop: read a coin name, use a switch expression to convert the Coin to its value

(ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.

(d) Stop the loop once the total reaches 15 or more.

(e) Print the change to return (total − 15).

Expected output: TEN then FIVE prints "Paid. Change: 0"; TEN then TEN prints

"Paid. Change: 5".

Code:

```
import java.util.Scanner;
 public class VendingMachine
{
     enum Coin
{
        ONE, TWO, FIVE, TEN
    }      public static void main(String[]
args) {

        Scanner sc = new Scanner(System.in);
         int price =
15;         int total =
0;
         while (total < price)
{

            System.out.print("Enter coin (ONE, TWO, FIVE, TEN):
");
            Coin coin = Coin.valueOf(sc.next().toUpperCase());
             int value =
0;
 switch (coin) {
case ONE:
value = 1;
break;
     case TWO:
value = 2;
break;
     case
FIVE:
        value = 5;
```

```
        break;
     case TEN:
value = 10;
break;
}               total +=
value;
            System.out.println("Total so far: " + total);
        }

        System.out.println("Paid. Change: " + (total - price));

sc.close();
    }
}
```

Output:



Aim:

Toll booth (TollBooth.java, main in this file)

(a) Define a record Vehicle(String number, String type).

(b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types "done" for the number: build a Vehicle, use a switch

expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type's

counter.

(d) After the loop, print the total toll and the type with the highest count. Expected output: two cars + one bike → "Total toll: 120", "Most frequent: car".

Code:

```
import java.util.Scanner;
 public class TollBooth {
     record Vehicle(String number, String type) {}
     public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
         int totalToll = 0;
int bikeCount = 0;
int carCount = 0;
int truckCount = 0;
         while (true) {

            System.out.print("Enter vehicle number (or done):
");
            String number = sc.next();
             if (number.equalsIgnoreCase("done")) {
break;
            }

            System.out.print("Enter vehicle type
(bike/car/truck): ");
            String type = sc.next().toLowerCase();

            Vehicle vehicle = new Vehicle(number, type);
             int toll = 0;
```

```
            switch (vehicle.type()) {
                 case "bike":
bikeCount++;
toll = 20;
break;
                 case "car":
                    carCount++;
toll = 50;                          break;
                 case "truck":
truckCount++;
toll = 150;
break;
                 default:
                    System.out.println("Invalid  vehicle
type.");                        toll = 0;
             }
              totalToll = totalToll + toll;
}

        System.out.println("\nTotal toll: " + totalToll);
        String mostFrequent;
         if (bikeCount >= carCount && bikeCount >= truckCount) {
mostFrequent = "bike";
        } else if (carCount >= bikeCount && carCount >=
truckCount) {              mostFrequent = "car";
        } else {
```

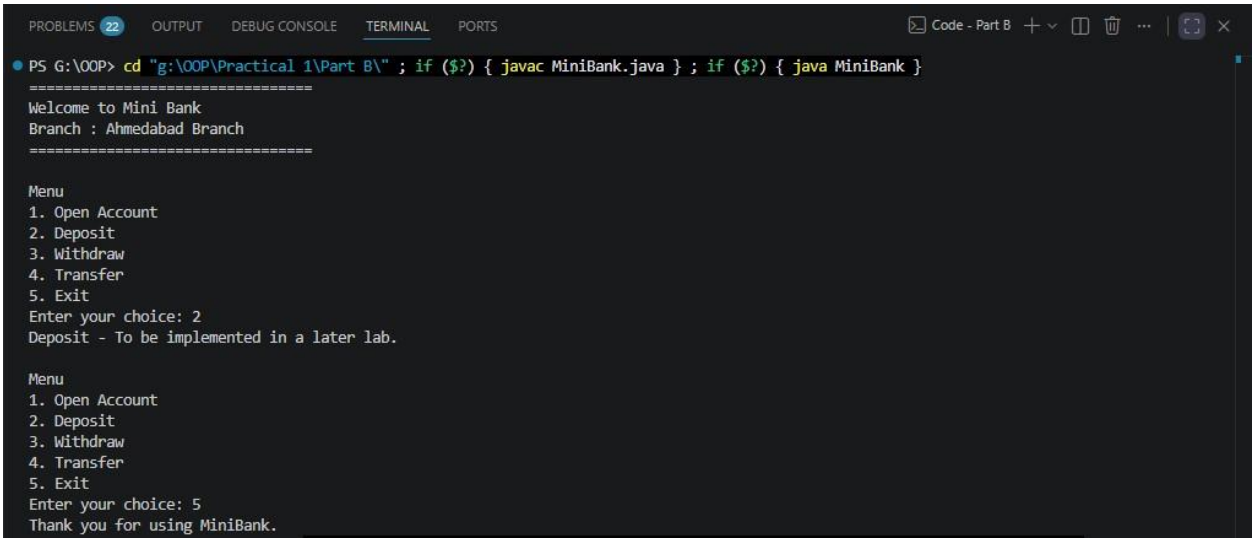
```
            mostFrequent = "truck";
        }

        System.out.println("Most frequent: " + mostFrequent);

sc.close();
    }
}
```

Output:




PROBLEMS 22
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code - Part B
PS G:\OOP> cd "g:\OOP\Practical 1\Part B\" ; if ($?) { javac MiniBank.java } ; if ($?) { java MiniBank }
==============================
Welcome to Mini Bank
Branch : Ahmedabad Branch
==============================
Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 2
Deposit - To be implemented in a later lab.
Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 5
Thank you for using MiniBank.

3. (For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)
(a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK.
(b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases).
(c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score.
(d) After 5 rounds, print the overall winner.
Expected output: each round prints both moves and the round winner; ends with a "You win 3–2" summary.

Code:

RPSLS.java

```
import java.util.Random;
import java.util.Scanner;

public class RPSLS {

    enum Move {
        ROCK, PAPER, SCISSORS, LIZARD, SPOCK
    }

    // Returns:
    //  1  -> a beats b
    // -1  -> b beats a
    //  0  -> tie
    static int winner(Move a, Move b) {

        if (a == b) {
            return 0;
        }

        return switch (a) {
            case ROCK -> (b == Move.LIZARD || b ==
Move.SCISSORS) ? 1 : -1;

            case PAPER -> (b == Move.ROCK || b == Move.SPOCK) ?
1 : -1;

            case SCISSORS -> (b == Move.PAPER || b ==
Move.LIZARD) ? 1 : -1;

            case LIZARD -> (b == Move.SPOCK || b == Move.PAPER)
? 1 : -1;
```

```
            case SPOCK -> (b == Move.SCISSORS || b == Move.ROCK)
? 1 : -1;
        };
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Random random = new Random();

        int playerScore = 0;
        int computerScore = 0;

        System.out.println("=== Rock-Paper-Scissors-Lizard-Spock
===");
        System.out.println("Enter one of: ROCK, PAPER, SCISSORS,
LIZARD, SPOCK");

        for (int round = 1; round <= 5; round++) {

            System.out.println("\nRound " + round);

            System.out.print("Your move: ");
            String input = sc.nextLine().trim().toUpperCase();

            Move playerMove;

            try {
                playerMove = Move.valueOf(input);
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid move! Please enter a
valid move.");
                round--;
```

```
                continue;
            }

            Move computerMove =
                    Move.values()[random.nextInt(Move.values().l
ength)];

            int result = winner(playerMove, computerMove);

            System.out.println("Your move     : " + playerMove);
            System.out.println("Computer move : " +
computerMove);

            if (result == 1) {
                System.out.println("Round winner: You");
                playerScore++;
            }
            else if (result == -1) {
                System.out.println("Round winner: Computer");
                computerScore++;
            }
            else {
                System.out.println("Round result: Tie");
            }

            System.out.println("Score: You " + playerScore
                    + " - " + computerScore + " Computer");
        }

        System.out.println("\n=== Final Result ===");

        if (playerScore > computerScore) {
            System.out.println("You win " + playerScore + "-"
                    + computerScore);
```

```
        }
        else if (computerScore > playerScore) {
            System.out.println("Computer wins " + computerScore
+ "-"
                    + playerScore);
        }
        else {
            System.out.println("It's a tie " + playerScore + "-"
                    + computerScore);
        }

        sc.close();
    }
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports

 === Rock-Paper-Scissors-Lizard-Spock ===
 Your move: ROCK
 Your move     : ROCK
 Computer move : ROCK
 Round result: Tie
 Score: You 0 - 0 Computer

 Round 2
 Your move: PAPER
 Your move     : PAPER
 Computer move : SPOCK
 Round winner: You
 Score: You 1 - 0 Computer

 Round 3
 Your move: SPOCK
 Your move     : SPOCK
 Computer move : LIZARD
 Round winner: Computer
 Score: You 1 - 1 Computer

 Round 4
 Your move: LIZARD
 Your move     : LIZARD
 Computer move : SPOCK
 Round winner: You
 Score: You 2 - 1 Computer

 Round 5
 Your move: ROCK
 Your move     : ROCK
 Computer move : LIZARD
 Round winner: You
 Score: You 3 - 1 Computer
```

Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is the role of the JVM, and what file does the javac compiler produce?

Ans.

- JVM (Java Virtual Machine) **runs Java** bytecode **on a computer.**
- **The `javac` compiler converts a Java source file (`.java`) into a** bytecode file (`.class`)**.**
- **The JVM then loads and executes the `.class` file.**

Flow:

```
.java source code
        ↓
     javac
        ↓
.class bytecode
        ↓
      JVM
        ↓
   Program runs
```

Example:

```
javac Hello.java
```

produces:

```
Hello.class
```

2. How does a switch expression differ from a traditional switch statement?

Ans.

Main differences

Switch Statement	Switch Expression
Performs statements/actions	Produces a value
Usually requires `break`	Uses `->`
Can cause fall-through	`->` cases don't fall through
Value must be assigned separately	Value can be assigned directly
Older Java feature	Modern Java feature

3. Why is an enum a good choice for a fixed set of menu options, and a record for read-only data?

Ans.

An enum is suitable when the possible values are a fixed, predefined set.

For example:

```
enum MenuOption {
    OPEN_ACCOUNT,
    DEPOSIT,
    WITHDRAW,
    TRANSFER,
    EXIT
}
```

This is better than using arbitrary strings such as `"deposit"` or `"withdraw"` because:

- **The allowed options are clearly defined.**
- **Invalid values are easier to prevent.**

- **Code becomes more readable.**
- **It works naturally with `switch`.**
- **Each option is a distinct type-safe value.**

Record

A record is useful for small objects whose purpose is primarily to hold read-only data.

For example:

```
record BankInfo(String name, String branch) {}
```

Java automatically provides:

- **A constructor**
- **Accessor methods such as `name()` and `branch()`**
- **`equals()`**
- **`hashCode()`**
- **`toString()`**

So instead of writing a full class just to store:

```
Bank name
Branch name
```

you can use a compact record.